



Open Computers

OpenComputers is a very powerful mod that allows the player to write/use custom scripts for nearly *anything* in GTNH. However, there is a steep learning curve and everything is in the Lua programming language. Not inherently a transport system, but drones/robots can be upgraded to handle transporting items and fluids. Not sure on TPS/FPS implications for servers/clients with large setups, or for potential speed. Standing blocks can substitute for a redstone control system. Requires significant infrastructure before useful operations, including power. Can be used to automate assembly line and infusion, but that will require quite a bit of coding. Basically advanced, technology based golems you have to program yourself. Could be push or pull based on your level of programming ability? May be regarded as meta-automation, that is - a system to control and interface with AE and redstone based automation setups.

Methods

Methods are built-in functions for specific machines. Connect to a machine by placing an adapter next to the controller, or by sneak right-clicking the controller with an MFU and inserting it into an adapter within 16 blocks. Use the first code block below to identify the name of the machine and then the second code block to list all the available methods for that machine.

1. Identify the name of the component (gt_machine, me_controller, etc).

```
local component = require('component')
for k,v in component.list() do print(k,v) end
```

2. Identify the methods for that specific component.

```
local component = require('component')
local machine = component.gt_machine
for k,v in pairs(machine) do print(k,v) end
```

Components

OpenComputers native components

Cookies help us deliver our services. By using our services, you agree to our use of cookies.

[More information](#)
[OK](#)

Some technical detail about ME system's material representation

When querying using some functions, OpenComputers will return table(s), each table representing an item stack, with the following specification:

```
{
  damage: number,
  hasTag: boolean,
  isCraftable: boolean,
  label: string,
  maxDamage: number,
  maxSize: number,
  name: string,
  size: number
}
```

It will also has more fields, depending on NBT of the item (when `hasTag == true`). Out of these, the ones that are most useful are `label`, which is the name of the item (name is unlocalized name, e.g. `minecraft:wool`, `IC2:itemDust2`), and `size`, which is the number of that item stored in ME network (`maxSize` is the number of stackable, e.g 64 for normal stack, 16 for eggs, ender pearls (though not in GTNH, only for example purposes)).

When there's a Fluid Discretizer in the ME system, fluids in the system are also represented by items (drops) and therefore can also be queried when querying items. There is a `fluidDrop` field added:

```
{
  damage: number,
  hasTag: boolean,
  isCraftable: boolean,
  label: string,
  maxDamage: number,
  maxSize: number,
  name: string,
  size: number,
  fluidDrop: {
    amount: number,
    label: string,
    name: string
  }
}
```

`fluidDrop.amount` is the same as `size`. However `fluidDrop.label` isn't the same as `label` (also happens for `fluidDrop.name` and `name`). `fluidDrop.label` is the name of the fluid, while `label` is "drop of " + the name of the fluid.

When querying fluids, they are represented as follow:

```
{
  hasTag: boolean,
  isCraftable: boolean,
  amount: number,
  label: string,
  name: string
}
```

It is the exact same as `fluidDrop` except that there are `hasTag` and `isCraftable`

Cookies help us deliver our services. By using our services, you agree to our use of cookies.

[More information](#)

Some functions allow for filtering with a table. When filtered with `{label="drop of Hydrochloric Acid"}`, it will return an array only contains item stack with drop of Hydrochloric Acid as label (if exists), which is Hydrochloric Acid in drop form. `{size=2}` will filter for item stacks with 2 in the system.

ME Controller

Function name	Function signature	Description
getItemsInNetwork	<code>function([filter:table]):table</code>	Get a list of the stored items in the network.
getFluidsInNetwork	<code>function():table</code>	Get a list of the stored fluids in the network.
getCpus	<code>function():table</code>	Get a list of tables representing the available CPUs in the network. These can't be used to issue crafting requests however. It's same as ME Terminal's Crafting Status where the player can see active items, final output, and can be used to cancel request.
getCraftables	<code>function([filter:table]):table</code>	Get a list of known item recipes. These can be used to issue crafting requests.

The `getCraftables` function will return a table with 2 functions: `getItemStack`, which will return a table representing an item stack of the crafting result, specified above, and `request([amount:int[, prioritizePower:boolean, cpuName: string]])`. The arguments are self-explanatory, with `amount` is the number of requested (defaults to 1), `prioritizePower` is whether the ME system will prioritize using "higher power" cpu, according to its comparison (out of all eligible cpus, choose the one with highest co-processors, then the one with highest crafting storage), with "lower power" is the reverse (defaults to true).

The `request` function will return another table that allows tracking the crafting status, with 4 functions `hasFailed`, `isCanceled`, `isComputing`, `isDone`. All of these 4 functions take no argument and return boolean so it is self-explanatory from here.

ME Interface

The ME Interface has all the functions listed in ME Controller section, on top of ME Interface-specific functions so it is stronger than ME Controller in terms of interfacing with the ME System.

Function name	Function signature	Description
getInterfaceConfiguration	<code>function([slot:number]):table</code>	Get the stored item config in the 9 slots of the interface.
getFluidInterfaceConfiguration	<code>function([slot:number]):table</code>	Same as getInterfaceConfiguration, but for fluid.
setInterfaceConfiguration	<code>function([slot:number][, database:address, entry:number[, size:number]]):boolean</code>	Config the 9 slots of the interface, requiring a database loaded with the item stack beforehand as it is hard to pass an item stack as an argument.
setFluidInterfaceConfiguration	<code>function(slot:number, index:number, database:address, entry:number):boolean</code>	Same as setInterfaceConfiguration, but for fluid.
storeInterfacePatternOutput	<code>function(slot:number, index:number, database:address, entry:number):boolean</code>	Store pattern output at the given index to the database entry.
storeInterfacePatternInput	<code>function(slot:number, index:number, database:address, entry:number):boolean</code>	Same as storeInterfacePatternOutput, but for input.
setInterfacePatternOutput	<code>function(slot:number, index:number, database:address, entry:number):boolean</code>	Set pattern output at the given index using the database entry, same reason as setInterfaceConfiguration.
setInterfacePatternInput	<code>function(slot:number, index:number, database:address, entry:number):boolean</code>	Same as setInterfacePatternInput, but for input.

With setInterfacePatternInput, setInterfacePatternOutput and the ability to request crafts, OpenComputers allows for a lot of meta-automation strategies: maintaining using OpenComputers, automating Magmatter and Degenerate Quark Gluon Plasma, to name a few.

Retrieved from "https://wiki.gtnewhorizons.com/wiki/Open_Computers?oldid=17556"

Cookies help us deliver our services. By using our services, you agree to our use of cookies.

More information